

GNSS-Denied Autonomy on a €500 Quadcopter

A companion-computer perception and planning stack for an ordinary analog FPV multirotor. Raspberry Pi 5, CPU only. No GPU, no ROS, no satellite fix.

COMPUTE	SENSOR	AUTOPILOT	LANGUAGE	VALIDATION
Raspberry Pi 5 8 GB, CPU only	Intel RealSense D435i	ArduPilot MAVLink v2	C++17 OpenCV, no ROS	Software-in-the-loop no flight yet

01 Summary

A €500 analog FPV quadcopter already flies well. It has an attitude controller that works, a pilot who can fly it, and a video link. What it cannot do is *decide where to go* when the radio link is jammed and the satellite fix is gone.

This project adds that decision to an unmodified airframe: a Raspberry Pi 5 bolted on, a depth camera, and a serial line to the flight controller. The engineering interest is not the hardware — every part is off the shelf. It is that the whole design is derived from one measured sensor limit, and that every subsequent choice can be traced back to it.

The document is organised around those choices. Section 03 states the constraint the sensor actually imposes; sections 04–06 are the architecture that follows from it; section 07 is the decision register; sections 08–09 are the evidence. Section 11 says what is not done.

MEASURED

A number behind it, produced by a run in this repository.

CODE

Read from source. Records what was written, not what worked.

ASSERTED

Believed, not tested. Treat as a hypothesis.

02

Constraints, fixed before anything was designed

These were not discovered during development. They were the starting conditions, and the architecture is what survives them.

CONSTRAINT	CONSEQUENCE IT FORCES
No satellite fix. Contested airspace takes GNSS first.	No global position anywhere in the flight path. Everything the planner uses must be body-frame and short-lived.
CPU only. A Pi 5 with no GPU and no accelerator.	Perception must fit in single-digit milliseconds per frame. Learned models are admissible only where a wrong answer is cheap.
The airframe is not modified. Stock ESC, stock FC, stock analog video.	The autopilot keeps the attitude loop. The Pi may only ask for a direction and a speed.
Attributable cost. The aircraft is expendable; the design has to be reproducible by anyone.	No LiDAR, no stereo module beyond a stock D435i, no custom PCB.
Analog video is the pilot's link and must stay untouched.	The Pi taps the existing composite signal passively. It adds zero latency to what the pilot sees.

03

The measurement that determines the architecture

A stereo depth camera does not have a range. It has a range *per question*. For the question this project asks — “*may I put a voxel of edge c here?*” — the answer holds only while the range error along the ray stays smaller than the cell:

$$Z_{\max} = \sqrt{(c \cdot f \cdot B / \sigma_d)} \times 0.75$$

On the D435i at 0.25 m cells this is **3.54 m**. **MEASURED** Not the camera's advertised range — its honest range for *this* representation.

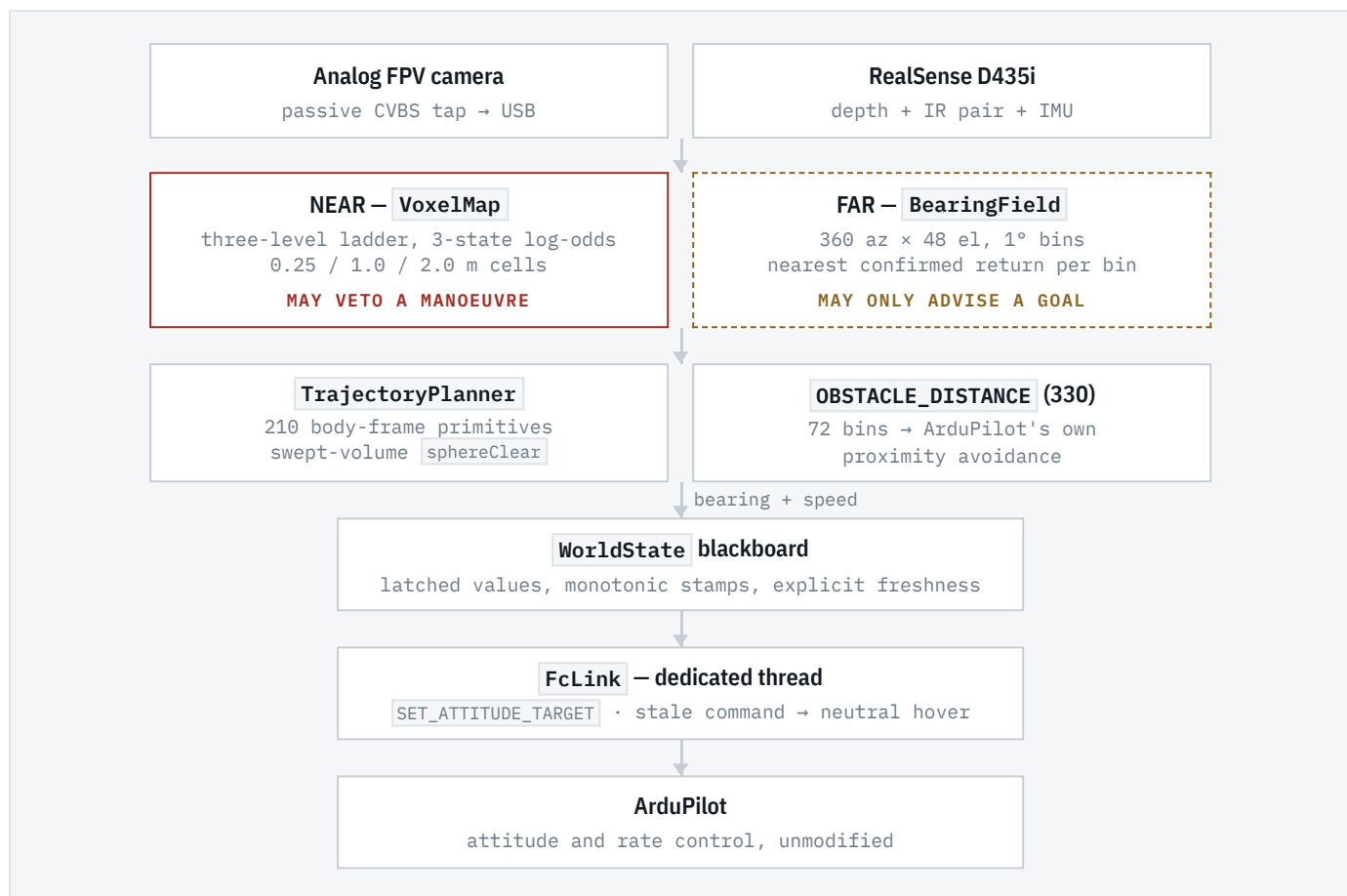
Past that distance the depth image still carries information. It simply is not information at voxel resolution, and the reason is that stereo error is **anisotropic and increasingly so**. Along the ray it grows quadratically; across the ray only linearly. Their ratio is $Z \cdot \sigma / B$:

$$10:1 \text{ at } 2 \text{ m} \quad \cdot \quad 100:1 \text{ at } 20 \text{ m}$$

A cube must be sized for the worse of the two axes. A voxel honest in range at 20 m would be 8 m wide, and would throw away the 4.5 cm of lateral detail the sensor still resolves at that distance. Two representations are therefore not a convenience; they are what the error model requires.

Cubes near. Bearings far.

Every other decision in this document is downstream of this one.



4.1 The near field: a resolution ladder, not a grid

A single cell size cannot be honest at both 2 m and 12 m, so the near field is three maps, each integrated only inside its own honest band and each consulted only there. Every level derives its own $Z_{\max}$ from the same formula rather than carrying a hardcoded number.

CODE

LEVEL	CELL	RAY STRIDE	MARKS OUT TO	CARVES OUT TO	ROLE
fine	0.25 m	1×	3.54 m	11.17 m	The only level allowed to veto
mid	1.00 m	1/4 rays	derived	25 m	Covers the gap the 8× jump would leave
far	2.00 m	1/16 rays	derived	40 m	Extends steering horizon only

Two details that are load-bearing rather than incidental.

Marking and carving are separate decisions with separate limits. Carving free space is bounded by the noise model at 11.17 m; marking occupied is bounded at 3.54 m. They are not the same question: erasing space you have seen through is safe much further out than asserting that something is there.

Level handovers sit at the exact marking range, with no overlap. A 2 m cell's near face can sit up to 2 m in front of the surface it contains, so consulting a coarse level inside a finer level's band draws obstacles far too close. Consulting levels by nearest-hit instead of by band

collapses the ladder to its coarsest rung — three levels integrated, one ever drawn. Both failures were observed on real hardware before the rule was written down. **MEASURED**

4.2 Three states, because unknown is not free

Occupancy is **FREE**, **OCCUPIED**, **UNKNOWN**. A two-state grid cannot represent “I have not looked there”, so it answers “*is this tube clear?*” with *yes* for space it has never observed.

This produced a real failure, not a hypothetical one. Scoring an unobserved direction as maximally open makes **straight up** the most attractive heading in the world, because the sky returns nothing. The aircraft climbed. The fix is one line and a symmetric penalty:

```
const float r = far->field->rangeAt(endAz, endEl);
farOpen = (r < 0.f) ? 0.f // unknown scores ZERO, not full reach
           : std::min(r, far->rangeM) / std::max(0.1f, far->rangeM);
```

The climb penalty is charged against the *commanded* elevation, not against level flight, and descent is penalised identically. A one-sided penalty does not remove the bias — it inverts it, and the aircraft dives instead. Maximum descent went from 0.10 m to 0.80 m before the symmetry was restored. **MEASURED**

4.3 Authority asymmetry

The two representations are deliberately **not** equal partners.

	NEAR FIELD	FAR FIELD
May do	Hard-reject a primitive whose swept 0.6 m ball touches an occupied cell	Contribute one weighted term to the score
Cost of an error	A manoeuvre is deleted. Enough of them and the aircraft stops.	A slightly worse heading.
Admits a learned model	No	Yes

Inventing awareness is cheap. Inventing permission is not.
This is why a monocular depth network is allowed to advise a direction and never to grant one.

4.4 The planner

A precomputed body-frame primitive library, rolled out and checked by swept volume. Precomputed because the aircraft is body-frame by construction: the library never changes, only the map it is tested against does. **CODE**

PARAMETER	VALUE	WHY THIS VALUE
Primitive count	210	13 yaw × 5 climb × 3 speed = 195 forward, plus 15 escape (5 of 8 rearward directions clearing a 60° off-nose gate × 3 climb)
Rollout horizon	2.0 s @ 0.1 s	Time, not distance, so a slow primitive is not judged over a longer path than a fast one
Robot radius	0.60 m	Configuration-space inflation — the vehicle is a point against a grown obstacle
Yaw lag τ	0.35 s	A commanded heading the airframe cannot reach is not a plan
Stopping model	3.0 m/s², 0.25 s react	Speed is capped so the aircraft can stop inside what it has actually mapped

SCORE TERMS, AND WHAT EACH IS BUYING

TERM	WEIGHT	BUYS
Goal alignment	1.00	Progress
Clearance along the primitive	0.70	Margin
Far-field openness at the endpoint	0.50	Not steering into a pocket the near map cannot yet see
Yaw-rate smoothness	0.25	Trackability
Climb / descent, symmetric	2.00 / 2.00	Refusing the free altitude the sky appears to offer

`sphereClear` is a hard gate applied before scoring: an occupied cell anywhere inside the swept ball rejects the primitive outright, regardless of how well it scores. Score terms trade against each other; the veto does not trade.

05 The flight-controller link

Three architectures were considered, and the choice between them is the strongest single decision in the project.

	WHO ESTIMATES POSITION	THE PI SENDS	VERDICT
A	The Pi; the autopilot consumes it	Synthetic GPS or vision pose	Retired
B	The autopilot; the Pi sends measurements	Odometry increments with covariance	Correct for v2
C	Nobody	Body-frame attitude commands	Correct for v1

Why A is not merely suboptimal but statistically wrong. ArduPilot's EKF3 treats every input as an independent measurement with a stated covariance. Feed it an estimate that has *already* been filtered on the Pi and you have two filters in series, neither aware of the other, and a covariance that no longer means anything. The failure is silent: the estimate looks plausible and is overconfident.

Why C is available at all. The thesis needs no global position. The planner is body-frame, the map is local and short-lived, and the output is a bearing and a speed. Under C there is no position estimate anywhere in the flight path — which also makes the GNSS-exclusion claim checkable rather than promised.

DOWNLINK — AUTOPILOT TO PI

MESSAGE	ID	CONSUMED FOR	STATUS
ATTITUDE	30	Gravity-align depth; de-rotate optical flow; rotate the bearing field	Decoded, angles and rates
HEARTBEAT	0	Mode, armed state, link health	Decoded
RC_CHANNELS	65	Assist switch, pilot intent	Decoded
SYS_STATUS	1	Battery → speed budget	Decoded
EKF_STATUS_REPORT	193	The autopilot's opinion of its own health	Decoded, not yet consumed
GPS_RAW_INT	24	Logged to the black box and routed nowhere else	Decoded

`EKF_STATUS_REPORT` closes a real gap in the safety argument: every mechanism in this system assumes the *map* might be wrong, and none assumes the *system* might be. The

autopilot's own estimator health is a free, independent second opinion that should gate the speed budget.

UPLINK — ATTITUDE, NOT VELOCITY

The consequence most likely to bite on a field day, stated in the design rather than discovered on the flight line: **GUIDED velocity setpoints require a horizontal velocity estimate, and without satellites there is not one.** No GPS, no external nav and no optical flow leaves EKF3 with inertial and barometric data — enough for attitude, not for velocity. `SET_POSITION_TARGET_LOCAL_NED` in a velocity mask therefore cannot work for v1, and the planner's speed is expressed as a pitch angle instead. Crude, and correct for the constraint.

Yaw is commanded absolutely, as the current heading plus the requested increment, so the command path **refuses to send without a fresh attitude** rather than guessing a heading. `CODE`

DEFENCE IN DEPTH, AT NO COST

`BearingField` emits `OBSTACLE_DISTANCE` (330): 72 distances by bearing, bin 0 at the nose, 65535 as the no-data sentinel, pinned by a golden-frame test. Publishing it enables a **second avoidance layer running inside ArduPilot** — different code, different failure modes, no dependence on our planner. For a system whose safety argument rests on independent paths to a veto, this is close to free.

06 Concurrency and the freshness doctrine

Perception results are **latched**, so a crashed or stalled worker thread leaves its last result set and readable. That is the right behaviour for control — and it is also a trap, because a latched value from a dead thread is indistinguishable from a live one.

The resolution is that freshness is explicit everywhere and never implied by a valid flag:

- Every result carries a monotonic timestamp written at production time.
- The fly loop stamps the tick once per cycle.
- Consumers call `corridorFresh(maxAge)`, never the bare flag.
- The link thread substitutes a **neutral hover** when the planner goes stale — it never repeats a stale *motion* command.
- Capture and inference share state by latest-value handoff, dropping frames rather than queueing them. A queue converts overload into latency; for a control loop, dropping is correct.

The same rule appears three times in three subsystems — `unknown ≠ free` in the map, `Fresh()` in the blackboard, neutral hover on the link. All three are the same statement: *absence of evidence is not evidence, and the system must be able to say so.*

07 Decision register

DECISION	CHOSEN	REJECTED, AND WHY
Flight stack	ArduPilot	Mature EKF3, documented GNSS-denied configuration, a proximity layer that consumes our output directly
Who estimates position (v1)	Nobody	Pi-side estimation double-filters against EKF3 and produces a meaningless covariance
Uplink interface	SET_ATTITUDE_TARGET	Velocity setpoints need a velocity estimate that does not exist without satellites
Near representation	3-state voxels	2-state cannot express “unobserved”, which is the one rule the safety argument rests on
Far representation	Angular bearing bins	Stereo anisotropy makes far cubes both wasteful and dishonest
Far-field authority	Advisory only	An invented obstacle must not be able to delete a manoeuvre
Memory model	Latch + stamp + explicit freshness	A latch from a dead thread looks live without a clock
Stale command	Neutral hover	Repeating a stale motion command is the worst available option
GNSS	Received, logged, routed nowhere	Makes the exclusion claim checkable by inspection instead of asserted
Coarse voxel rung beyond 12 m	Retired	Bearings are 33× finer for less compute
0.10 m near rung	Retired	Cost 37% of the frame budget <i>and</i> reduced near coverage — 76.3% against 100% at 2 m standoff
Learned far-field depth	Parked	Monocular depth measured close-field-only: inverse-depth output loses resolution as $1/Z^2$, and per-frame normalisation lets one near object crush the far field

08 Evidence: two planners, measured against each other

Six worlds × eight seeds, paired. **MEASURED**

- **Trajectory planner** — asks “*is this entire 2 s swept arc clear, 0.6 m ball at every point?*”
- **Histogram planner** — VFH-style; asks “*does this bearing look open?*”

The histogram planner wins progress in **all six worlds, significantly**: paired |t| from 2.76 to 17.71, with the trajectory planner winning 0 or 1 of 8 seeds everywhere. Taken alone that reads as a verdict. It is not.

The trajectory planner is not blocked, it is throttled. Instrumented, 195–207 of 210 primitives reject on *genuinely mapped* occupied cells, with free distance correctly near zero and speed correctly zero. Indoors the 2 s rollout is simply longer than the room. The histogram planner never stops because it never asks for a tube — it steers. It is checking less, and that is the entirety of its advantage.

Where the extra check earns its cost, it shows up as clearance, and only in the two genuinely enclosed worlds:

WORLD	CLEARANCE Δ (TRAJ - HIST)	T	TRAJ WINS
forest	+0.035 m	0.34	1/8
lanes	+0.110 m	1.28	5/8
city	-0.016 m	0.08	3/8
road	-0.101 m	1.49	2/8
indoor	+0.120 m	3.41	7/8
cul-de-sac	+0.605 m	3.69	6/8

Collisions: 1 in 96 runs. Compute: the trajectory planner is **2–3× cheaper** — 2.2 ms against 7.1 ms in forest. Neither planner was retired; the measurement changed what each is *for*, not which one survives.

A RESULT THAT REVERSED UNDER SAMPLE SIZE

At $n=3$ the far-field term appeared to be hurting the planner, and I said so. At $n=48$ the sign is the opposite: turning the far field off improves progress everywhere and cuts stopping sharply — but in the city it produced **two collisions where the far field on produced none**, plus a significant clearance loss in the cul-de-sac. It is not a defect. It is buying safety with progress, which is the trade the entire planner makes.

Called a bug on three samples; a safety feature on forty-eight.

The reason every claim in this document carries an evidence tag.

09 Evidence: motion estimation needs two sensors

The D435i carries an IMU. It does **not** compute optical flow — its depth ASIC does stereo matching and nothing else. What it does provide is two rectified IR streams and per-pixel depth from the same instant, which removes both hard parts of computing flow yourself.

INERTIAL ALONE FAILS, AND NO BETTER IMU REPAIRS IT

Short-memory inertial odometry integrates between zero-velocity updates and declares itself invalid once predicted drift exceeds budget. Drift is dominated by attitude error, not by sensor noise:

$$\text{drift} \approx \frac{1}{2} \cdot g \cdot \sin(\theta_{\text{err}}) \cdot t^2$$

Measured, it fails at every tilt error tested: the legs the planner actually flies are 3.70 s against a 1.71 s budget at 1° of attitude error. **MEASURED** Inertial error grows as t^2 ; velocity-measured error grows as t . That is a difference in *order*, so no component upgrade closes it — only a velocity measurement does.

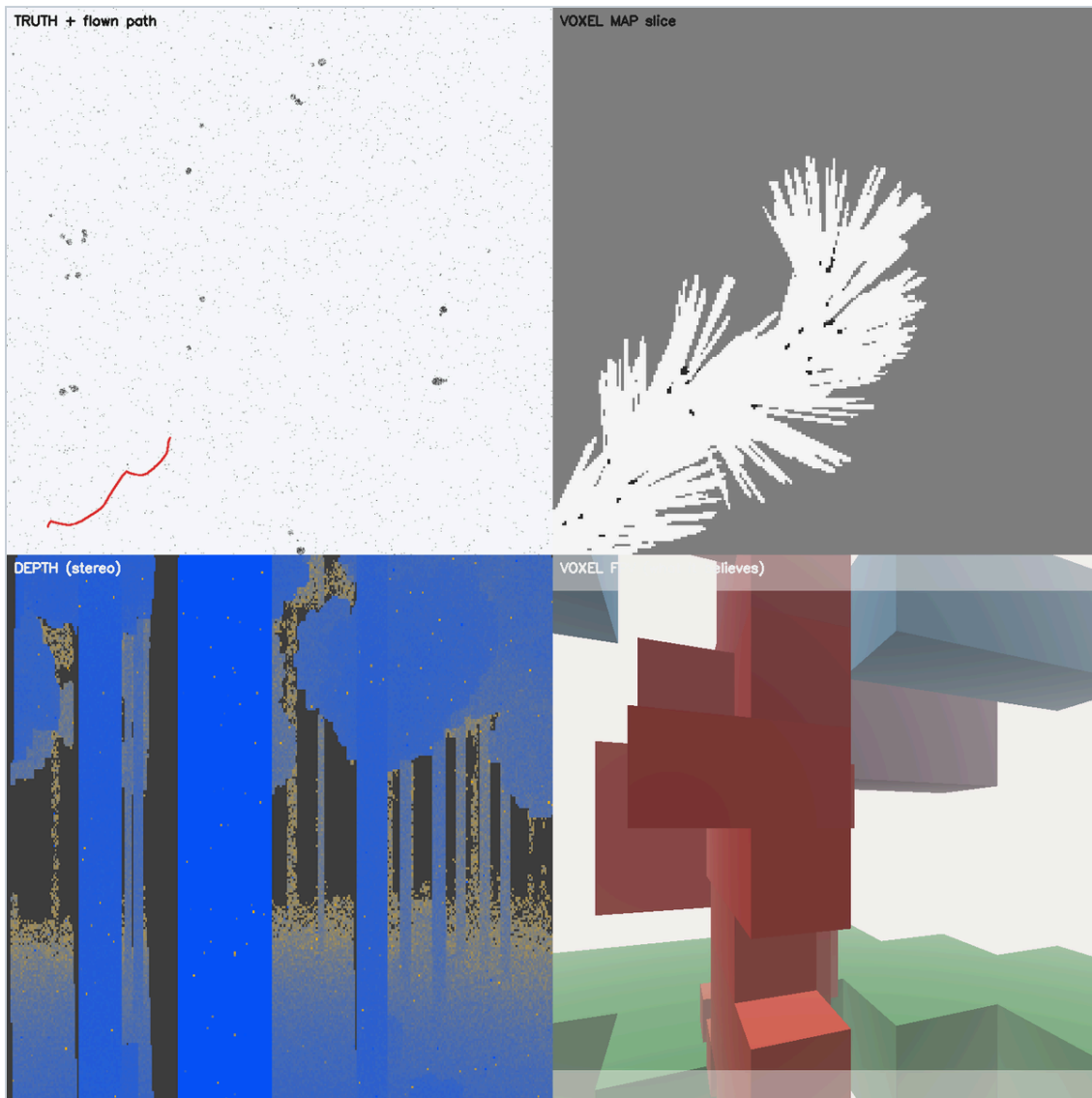
FLOW × DEPTH SUPPLIES THE MISSING MEASUREMENT

Sparse grid match on the IR stream, forward-backward gated, de-rotated using the autopilot's attitude, scaled by per-point depth, solved as a 3×3 least squares. Fused with the IMU: **4.1× mean and 6.9× worst-case improvement**. **MEASURED**

The instructive part is what did *not* matter. Flow **quality** barely moved the result — 5% against 15% noise changed it by 22%. Flow **presence** dominated. The design question is not how accurate the flow is; it is whether there is any flow at all, which makes texture yield the variable worth measuring and accuracy the one worth ignoring.

A textureless wall yields no estimate, rather than a confident zero.

A confident zero is the lethal failure: the estimator believes it is stationary and integrates its own drift as truth.

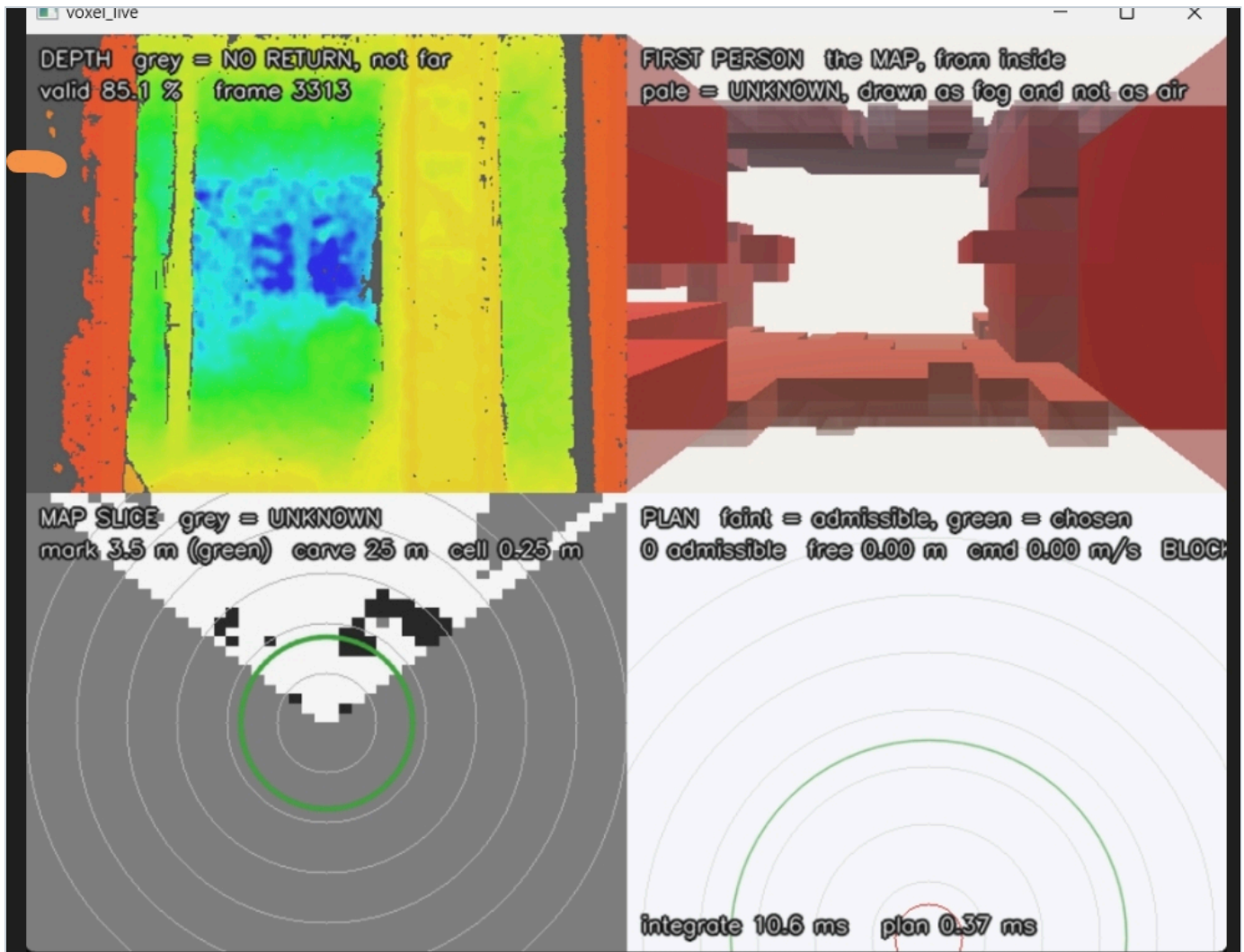


Forest — the design case, and the reference for reading every view of this kind.

Top left, ground truth with the flown path — visible to the scorer and never to the planner. *Top right*, a horizontal slice of the map: white is observed free, grey is unknown, dark is occupied. *Bottom left*, the stereo depth frame. *Bottom right*, the aircraft's own first-person render of its map — the only pane the planner can see.

Colour in the first-person pane is height relative to the aircraft, not object class: green is roughly 3.5 m below, red is at its own altitude, blue above, with per-face shading so a cube grid reads as geometry rather than noise. Two further cues are deliberate. Distance hazes surfaces toward the background colour, so depth is legible without a number; and **uncertainty fog** washes out any surface seen *through* unknown space in proportion to how much was traversed. A wall behind four metres of unobserved volume is a guess, and it is drawn as one.

The split between the top-left and bottom-right panes is the point of the whole harness. The aircraft plans against its own noisy map; collisions are scored against ground truth. They are different data structures on purpose — a harness that checked the plan against the map it planned with would pass no matter how wrong the map was.



The same pipeline on real sensor data, handheld indoors.

Measured depth at 85.1% valid; integrate 10.6 ms, plan 0.37 ms. Pale is unknown, drawn as fog and not as air — the void ahead is not open space, it is space that has not been observed. The plan pane reads **0 admissible, 0.00 m/s, BLOCKED**. Given a map it has not earned, the correct output is to stop, and it stops.

11 Verification, status, and what is not done

HOW IT IS VERIFIED

- **Ground truth is a separate data structure from the map.** Collisions are detected against the world, never against the aircraft's belief.
- **A perfect-depth control run** accompanies every sensor run. If it fails with perfect depth the planner is broken; if it passes with perfect depth and fails with stereo, the number is a sensor result rather than a planner result.
- **Wire formats are pinned against golden frames** generated by an independent implementation (pymavlink), not against our own encoder.
- **12 registered checks in the simulation tree**, all passing, re-run for this document; **13 test programs** in the flight runtime tree.
- **Multi-seed, paired statistics** for any claim comparing two configurations. Single-seed results are treated as anecdotes, because one of them already reversed.

■ Built and measured

- Three-level 3-state voxel ladder, 6.8–9.6 ms/frame
- 360×48 bearing field, 2.5 ms, yaw as an exact index shift
- 210-primitive planner with swept-volume veto, 2.2 ms
- Histogram planner, retained as a comparator
- MAVLink v2 codec, golden-frame pinned
- Six downlink decoders; attitude uplink with a refusal path
- `OBSTACLE_DISTANCE` producer
- Inertial odometry with zero-velocity and velocity updates
- Flow-to-metric-velocity estimator
- Six simulated worlds: forest, lanes, city, road, indoor, cul-de-sac

■ Not done

- 01 **Nothing has run on a Pi 5.** Every timing here is development-machine. A Cortex-A76 on memory-bound work is plausibly 1.5–2.5× slower `ASSERTED`. This reprises every plan in the project and it is an afternoon's work.
- 02 **The depth camera has never flown.** All real captures are handheld.
- 03 **Flow yield on real bark and foliage is unmeasured.** Correctness is settled; yield is not, and section 09 shows yield is the variable that decides the outcome.
- 04 **The IR stream is defined but never requested** in the camera layer, so the flow estimator currently has no frames to consume.
- 05 **Two world models share no code** — one 0.5 m 2-state with a 1.5 m robot radius, one 0.25 m 3-state at 0.35 m. Three angular resolutions and two radii differing by 4× is an accident of two codebases, and the 2-state one cannot express the rule the safety argument rests on.
- 06 **The `OBSTACLE_DISTANCE` publisher exists only in the simulation tree, not in the flight runtime.**
- 07 **Autopilot estimator health is decoded but not consumed.**
- 08 **Retired architecture-A fields remain** in the world state. They are correct code for architecture B and should be marked unwired rather than deleted.
- 09 **The non-satellite autopilot configuration is not written down.** GUIDED will refuse to engage until it is, and the symptom presents as a broken link.

Items 1–4 are what stands between this and a first flight, and none is large. Items 5 and 8 are architecture debt. Item 3 is the only one that could invalidate a design decision rather than merely delay it, which is why it is scheduled first among the hardware tasks.

Scope. This is a working technical demonstrator built to explore what is achievable on cheap, CPU-only hardware. It is not a certified or commercially flown system, and validation status is stated explicitly throughout rather than summarised favourably.

C++17 · OpenCV · MAVLink v2 · no ROS, no GPU, no CUDA. Simulation harness, flight runtime, and perception prototyping tools are separate trees in one repository.